

Ex. No.: 6a

Experiment Title: First Come First Serve (FCFS) Scheduling

Aim:

To implement the First-Come First-Serve (FCFS) scheduling algorithm.

Algorithm:

1. Start the program and input the number of processes.
2. For each process, input the burst time.
3. Initialize the waiting time and turnaround time.
4. For each process:
 - Waiting time for first process is 0.
 - For others, it is the sum of burst times of all previous processes.
5. Turnaround time = Waiting time + Burst time for each process.
6. Calculate total waiting time and total turnaround time.
7. Display process details: burst time, waiting time, and turnaround time.
8. Compute average waiting and turnaround times.
9. End.

Program Code:

```
#include <stdio.h>
```

```
int main() {
```

```

int n, i;
int bt[20], wt[20], tat[20];
float avg_wt = 0, avg_tat = 0;

printf("Enter the number of processes: ");
scanf("%d", &n);

printf("Enter the burst time of the processes:\n");
for(i = 0; i < n; i++) {
    scanf("%d", &bt[i]);
}

wt[0] = 0;
for(i = 1; i < n; i++) {
    wt[i] = wt[i-1] + bt[i-1];
}

for(i = 0; i < n; i++) {
    tat[i] = bt[i] + wt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("Process\tBurst Time\tWaiting Time\tTurn Around Time\n");
for(i = 0; i < n; i++) {
    printf("%d\t%d\t\t\t%d\t\t\t%d\n", i, bt[i], wt[i], tat[i]);
}

printf("Average waiting time is: %.1f\n", avg_wt / n);
printf("Average Turn around Time is: %.1f\n", avg_tat / n);

return 0;
}

```

Sample Output:

Enter the number of process:

3

Enter the burst time of the processes:

24 3 3

Process	Burst Time	Waiting Time	Turn Around Time
0	24	0	24
1	3	24	27
2	3	27	30

Average waiting time is: 17.0

Average Turn around Time is: 27.0

Result:

The program was executed successfully. It calculated the waiting time and turnaround time for each process using FCFS scheduling.

Ex. No.: 6b

Date: *[Write the date here]*

Experiment Title: Shortest Job First (SJF) Scheduling

Aim:

To implement the Shortest Job First (SJF) scheduling algorithm.

Algorithm:

1. Start the program and declare a structure to store process details (process ID, burst time, waiting time, turnaround time).
2. Input the number of processes.
3. Read the burst time for each process and assign process IDs.
4. Initialize waiting time and turnaround time to zero.
5. Sort the processes based on their burst time in ascending order.
6. For each process, calculate:
 - $\text{Waiting Time} = \text{Sum of burst times of all previous processes.}$
 - $\text{Turnaround Time} = \text{Waiting Time} + \text{Burst Time.}$
7. Calculate total and average waiting and turnaround times.
8. Display the process details along with the averages.
9. End.

Program Code:

```
#include <stdio.h>
```

```
struct process {  
    int pid;  
  
    int bt;  
  
    int wt;  
  
    int tat;  
};  
  
void sort(struct process p[], int n) {  
    struct process temp;  
    for(int i = 0; i < n-1; i++) {  
        for(int j = i+1; j < n; j++) {  
            if(p[i].bt > p[j].bt) {  
                temp = p[i];  
                p[i] = p[j];  
                p[j] = temp;  
            }  
        }  
    }  
}
```

```
int main() {  
    int n;  
  
    float avg_wt = 0, avg_tat = 0;
```

```
printf("Enter the number of process: ");

scanf("%d", &n);


struct process p[n];


printf("Enter the burst time of the processes:\n");

for(int i = 0; i < n; i++) {

    p[i].pid = i + 1;

    scanf("%d", &p[i].bt);

}


sort(p, n);


p[0].wt = 0;

p[0].tat = p[0].bt;

for(int i = 1; i < n; i++) {

    p[i].wt = p[i-1].wt + p[i-1].bt;

    p[i].tat = p[i].wt + p[i].bt;

}


printf("Process\tBurst Time\tWaiting Time\tTurn Around Time\n");

for(int i = 0; i < n; i++) {
```

```

        printf("%d\t%d\t\t%d\t\t%d\n", p[i].pid, p[i].bt, p[i].wt,
p[i].tat);

        avg_wt += p[i].wt;

        avg_tat += p[i].tat;

    }

    printf("Average waiting time is: %.1f\n", avg_wt / n);

    printf("Average Turn Around Time is: %.1f\n", avg_tat / n);

    return 0;
}

```

Sample Output:

Enter the number of process:

4

Enter the burst time of the processes:

8 4 9 5

Process	Burst Time	Waiting Time	Turn Around Time
2	4	0	4
4	5	4	9
1	8	9	17
3	9	17	26

Average waiting time is: 7.5

Average Turn Around Time is: 13.0

Result:

The program was executed successfully. It calculated the waiting time and turnaround time for each process using the SJF scheduling algorithm.

Ex. No.: 6c

Experiment Title: Priority Scheduling

Aim:

To implement priority scheduling technique.

Algorithm:

1. Start the program and declare a structure to store process details (process ID, burst time, priority, waiting time, turnaround time).
2. Input the number of processes.
3. Read burst time and priority for each process and assign process IDs.
4. Sort the processes based on **priority** in ascending order (lower number = higher priority).
5. Initialize the first process's waiting time to 0.
6. For each process:
 - Waiting Time = Sum of burst times of all previous processes.
 - Turnaround Time = Waiting Time + Burst Time.
7. Calculate total and average waiting time and turnaround time.
8. Display the process details along with the averages.
9. End.

Program Code:

```
c
CopyEdit
#include <stdio.h>
```

```

struct process {
    int pid;
    int bt;
    int priority;
    int wt;
    int tat;
};

void sort(struct process p[], int n) {
    struct process temp;
    for(int i = 0; i < n-1; i++) {
        for(int j = i+1; j < n; j++) {
            if(p[i].priority > p[j].priority) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

int main() {
    int n;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter the number of process: ");
    scanf("%d", &n);

    struct process p[n];

    printf("Enter burst time and priority for each process:\n");
    for(int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Process %d:\n", i + 1);
        printf("Burst time: ");
        scanf("%d", &p[i].bt);
        printf("Priority: ");
    }
}

```

```

        scanf("%d", &p[i].priority);
    }

    sort(p, n);

    p[0].wt = 0;
    p[0].tat = p[0].bt;

    for(int i = 1; i < n; i++) {
        p[i].wt = p[i-1].wt + p[i-1].bt;
        p[i].tat = p[i].wt + p[i].bt;
    }

    printf("Process\tBurst Time\tPriority\tWaiting Time\tTurn Around Time\n");
    for(int i = 0; i < n; i++) {
        printf("%d\t%d\t\t\t%d\t\t\t%d\t\t\t%d\n", p[i].pid, p[i].bt, p[i].priority, p[i].wt, p[i].tat);
        avg_wt += p[i].wt;
        avg_tat += p[i].tat;
    }

    printf("Average Waiting Time: %.2f\n", avg_wt / n);
    printf("Average Turn Around Time: %.2f\n", avg_tat / n);

    return 0;
}

```

Sample Output:

yaml

CopyEdit

Enter the number of process:

4

Enter burst time and priority for each process:

Process 1:

Burst time: 10

Priority: 3

Process 2:
Burst time: 5
Priority: 1
Process 3:
Burst time: 8
Priority: 2
Process 4:
Burst time: 6
Priority: 4

Process	Burst Time	Priority	Waiting Time	Turn Around Time
2	5	1	0	5
3	8	2	5	13
1	10	3	13	23
4	6	4	23	29

Average Waiting Time: 10.25
Average Turn Around Time: 17.50

Result:

The priority scheduling algorithm was implemented successfully. It calculated the waiting time and turnaround time based on process priorities.

Ex. No.: 6d

Experiment Title: Round Robin Scheduling

Aim:

To implement the Round Robin (RR) scheduling technique.

Algorithm:

1. Start the program and declare arrays to store process data and timing values.
2. Input number of processes and time quantum.
3. Read burst time for each process.
4. Copy burst times into `rem_bt[]` to track remaining burst times.
5. Initialize `wt[]` (waiting time) array to 0.
6. Initialize time variable `t = 0`.
7. While all processes are not completed, repeat:
 - For each process `i`:
 - If `rem_bt[i] > quantum`:
 - Increment time `t += quantum`
 - Decrease `rem_bt[i] -= quantum`
 - Else if `rem_bt[i] > 0`:
 - `t += rem_bt[i]`
 - `wt[i] = t - bt[i]`
 - `rem_bt[i] = 0` (process complete)

8. For each process, calculate $tat[i] = wt[i] + bt[i]$.
9. Compute average waiting time and average turnaround time.

Program Code:

c

CopyEdit

```
#include <stdio.h>

int main() {
    int n, i, quantum, t = 0, flag;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rem_bt[n], wt[n], tat[n];

    printf("Enter burst times for each process:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
    }
```

```
        wt[i] = 0;
    }

    printf("Enter Time Quantum: ");

    scanf("%d", &quantum);

    do {
        flag = 0;

        for(i = 0; i < n; i++) {
            if(rem_bt[i] > 0) {
                flag = 1;

                if(rem_bt[i] > quantum) {
                    t += quantum;

                    rem_bt[i] -= quantum;
                } else {
                    t += rem_bt[i];

                    wt[i] = t - bt[i];

                    rem_bt[i] = 0;
                }
            }
        }
    }

    } while(flag);
```

```

for(i = 0; i < n; i++) {
    tat[i] = bt[i] + wt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nProcess\tBurst Time\tWaiting Time\tTurn Around Time\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t\t%d\t\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time: %.2f", avg_wt / n);
printf("\nAverage Turn Around Time: %.2f\n", avg_tat / n);

return 0;
}

```

Sample Output:

```

Enter number of processes:
4
Enter burst times for each process:
P1: 10

```


P2: 5

P3: 8

P4: 6

Enter Time Quantum: 3

Process		Burst Time	Waiting Time	Turn Around Time
P1	10	15	25	
P2	5	9	14	
P3	8	14	22	
P4	6	12	18	

Average Waiting Time: 12.50

Average Turn Around Time: 19.75

Result:

The Round Robin Scheduling algorithm was implemented successfully. The waiting and turnaround times were calculated using a fixed time quantum.